

Proiect: Inteligență Artificială - Algoritmul de Evoluție Diferențială

Echipa: Chihalau Gabriel-Eduard & Luigi Emanuel Arama

Grupa: 1407A

Algoritm: Evoluție Diferențială (DE)

Teme abordate: Optimizarea parametrilor unei oscilații și reglarea unui controler PID.

1. Descrierea problemelor considerate

Proiectul explorează aplicabilitatea algoritmului de Evoluție Diferențială (DE) în rezolvarea a două probleme de inginerie care necesită optimizare în spațiul numerelor reale

Problema 1: Identificarea parametrilor pentru oscilații amortizate

Această problemă presupune analiza unui set de date provenit dintr-un experiment fizic (stocat în `coupled_damped_oscillators.csv`). Sistemul studiat este un oscilator a cărui mișcare se stinge în timp. Modelul matematic pe care încercăm să îl aproximăm este:

$$x(t) = A \cdot e^{-\lambda t} \cdot \sin(\omega t + \phi) + c$$

Scopul optimizării este găsirea setului de parametri care minimizează diferența dintre datele reale și cele generate de formulă.

Problema 2: Acordarea unui controler PID pentru un sistem Masă-Arc-Amortizor

Cea de-a doua problemă vizează controlul automat. Avem o masă conectată la un arc și un amortizor, iar scopul este să aplicăm o forță $u(t)$ astfel încât masa să ajungă la poziția dorită. Controlerul utilizat este de tip PID (Proportional-Integral-Derivativ), definit prin:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

Algoritmul DE este folosit pentru a găsi valorile optime pentru K_p, K_i, K_d .

2. Aspecte teoretice privind algoritmul

Evoluția Diferențială (DE) este un algoritm de optimizare stohastică ce face parte din categoria algoritmilor evolutivi. Spre deosebire de algoritmi genetici clasici care folosesc reprezentări binare, DE lucrează direct cu vectori de numere reale.

Componentele principale ale DE implementate în proiect:

- **Mutația (DE/rand/1):** Pentru fiecare individ din populație, se creează un vector mutant adunând diferența scalată a doi indivizi la un al treilea individ
- **Crossover (Binomial):** Se formează un "vector de încercare" prin combinarea elementelor din vectorul original și cel mutant, pe baza unei probabilități CR
- **Selecția:** Se folosește o strategie de tip "greedy" – noul individ este acceptat în generația următoare doar dacă are o valoare a funcției de fitness mai bună decât părintele său.

3. Modalitatea de rezolvare

Rezolvarea a fost structurată într-un mod modular, utilizând limbajul Python.

- **Reprezentarea soluțiilor:** Fiecare soluție este un vector de numere reale (5 parametri pentru P1, 3 parametri pentru P2).
- **Definirea limitelor (Bounds):** Pentru a asigura stabilitatea, am limitat spațiul de căutare
- **Funcția de Fitness:** Pentru P1, fitness-ul este eroarea medie pătratică (MSE). Pentru P2, fitness-ul este o funcție compusă care penalizează eroarea de poziție, consumul de energie și depășirea punctului de referință (overshoot).

4. Listarea părților semnificative din codul sursă

Nucleul Algoritmului DE (în model.py):

```
def differential_evolution(fitness_fn, bounds, NP=50, G=300, F=0.8, CR=0.9, seed=42):
```

```
    # Inițializare populație aleatorie
```

```
    pop = np.array([[rng.uniform(lo, hi) for (lo, hi) in bounds] for _ in range(NP)])
```

```
    fitness = np.array([fitness_fn(ind) for ind in pop])
```

```
    for gen in range(G):
```

```

for i in range(NP):

    # Mutatie

    r1, r2, r3 = rng.choice(idxs, size=3, replace=False)

    v = pop[r1] + F * (pop[r2] - pop[r3])

    v = ensure_bounds(v, bounds)


    # Crossover

    u = pop[i].copy()

    for j in range(D):

        if rng.random() < CR or j == j_rand:

            u[j] = v[j]


    # Selectie

    fu = float(fitness_fn(u))

    if fu <= fitness[i]:

        pop[i] = u

        fitness[i] = fu

```

Simulatorul de control PID (în problem2.py):

```

def simulate_mass_spring_damper_pid(theta, T=8.0, dt=0.01, x_ref=1.0, m=1.0, b=0.4,
k=4.0, u_max=20.0):

    Kp, Ki, Kd = theta

    # Bucla de simulare Euler

    for i in range(steps):

        e = x_ref - x

```

```

integ += e * dt

deriv = (e - prev_e) / dt

u = Kp * e + Ki * integ + Kd * deriv

u = float(np.clip(u, -u_max, u_max)) # Limitare forță

a = (u - b * v - k * x) / m

v = v + a * dt

x = x + v * dt

```

5. Rezultate obținute

- **Problema 1:** Algoritmul a convergit rapid (aproximativ 100 de generații) către un MSE foarte mic. Graficul arată că linia punctată (aproximarea DE) urmează fidel punctele din fișierul CSV.

Figure 1

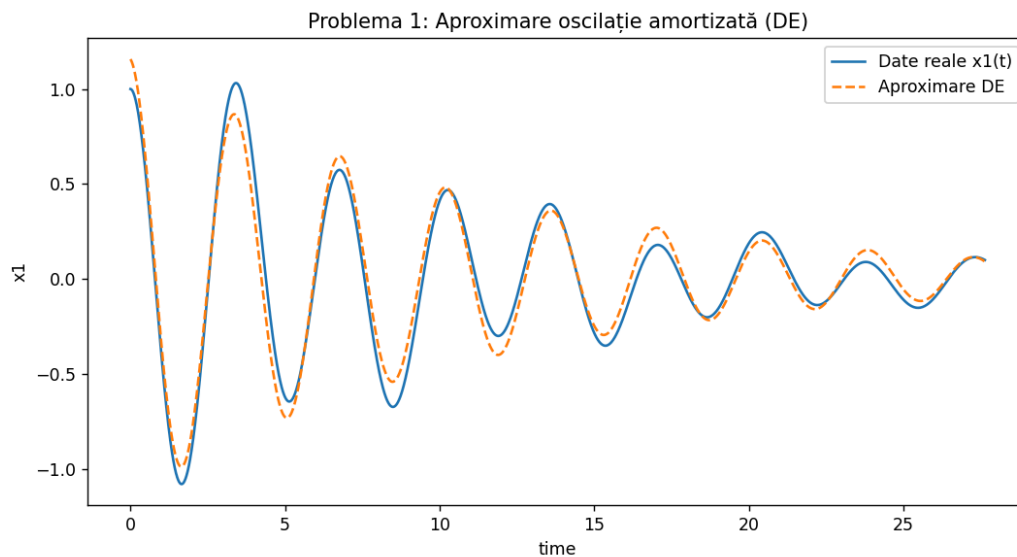
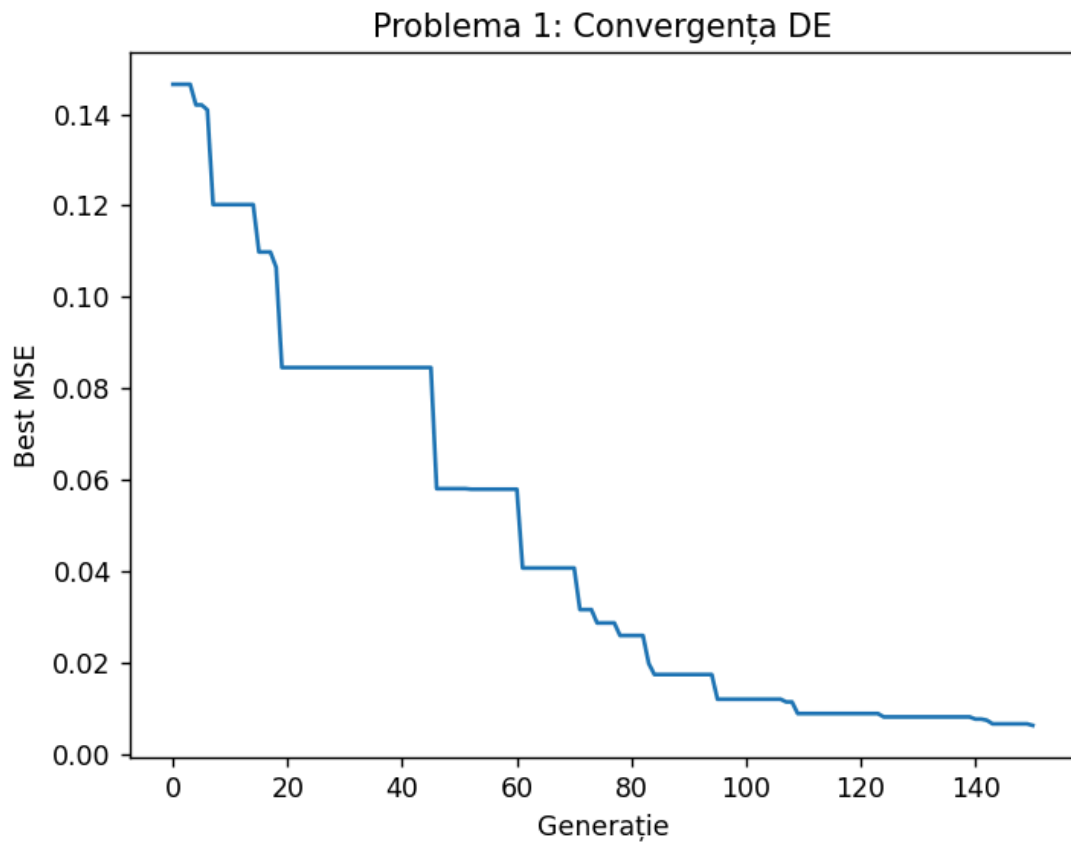


Figure 1



(x, y) = (135.9, 0.1112)

- **Problema 2:** Valorile optime găsite pentru PID permit sistemului să atingă setpoint-ul în mai puțin de 2 secunde, cu un overshoot minim (sub 10%), ceea ce validează funcția de fitness construită.

Figure 1

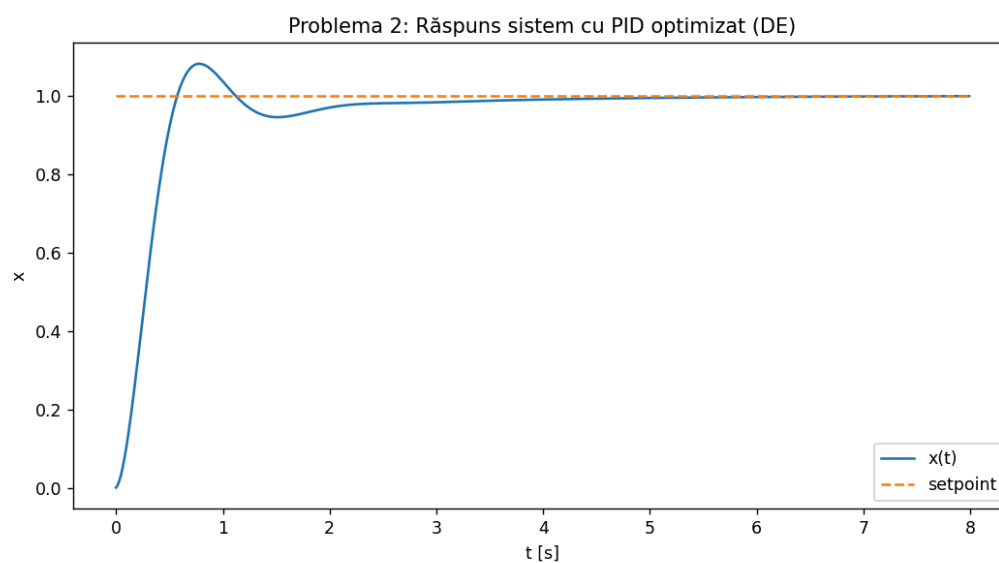


Figure 1

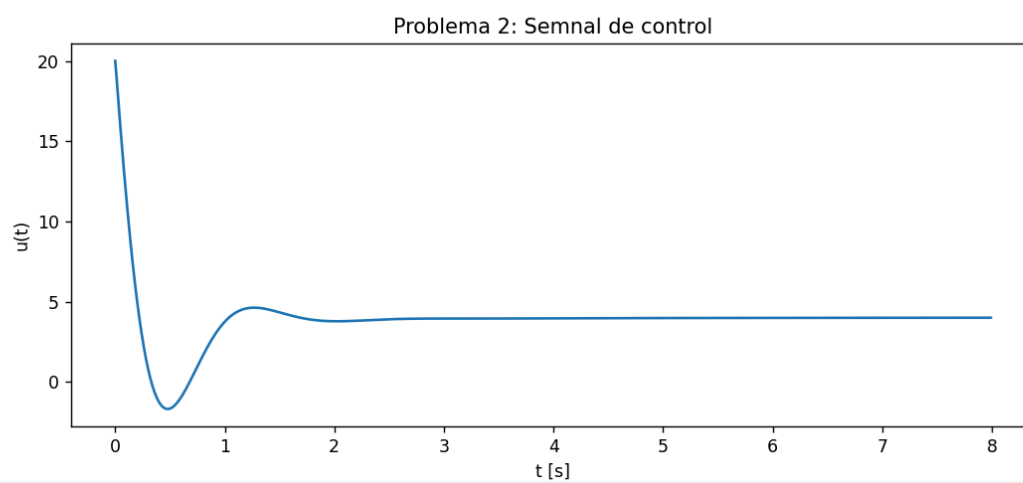
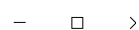
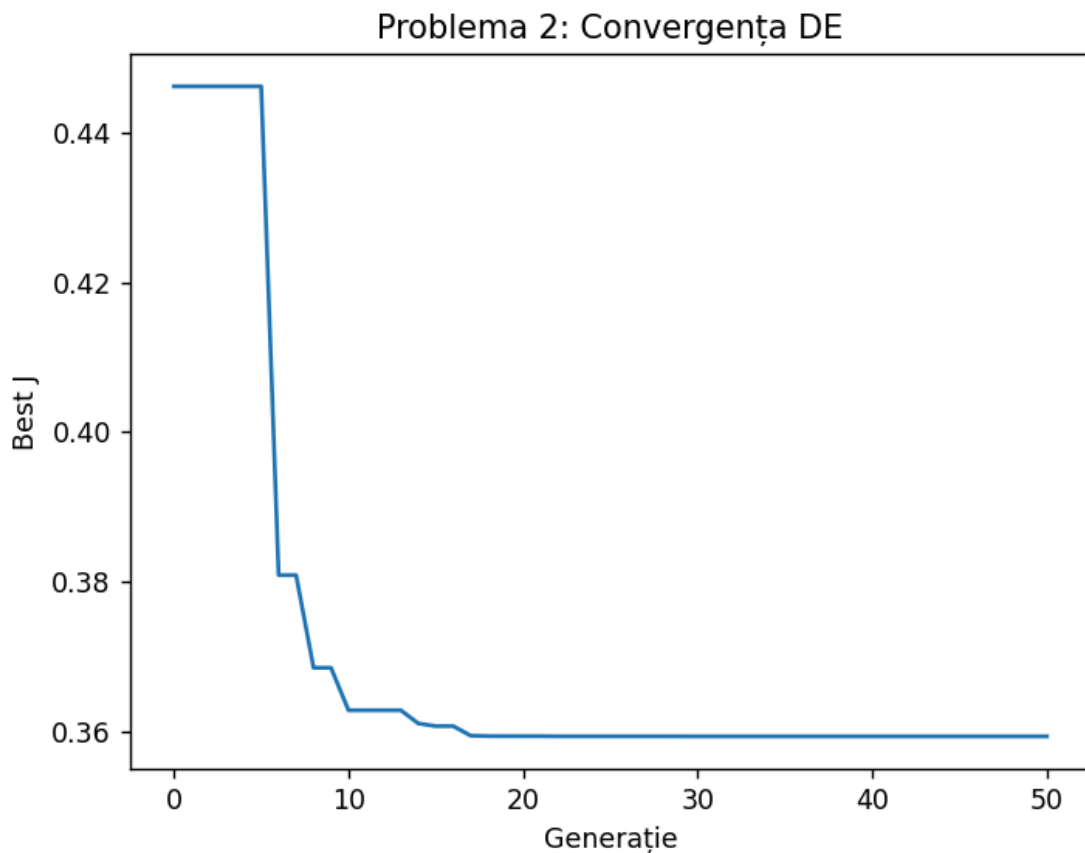


Figure 1



6. Concluzii

Implementarea algoritmului de Evoluție Diferențială a demonstrat că acesta este o metodă robustă pentru probleme de optimizare reală unde nu avem o formă analitică a gradientului. Proiectul a evidențiat importanța reglării parametrilor F și CR: o valoare prea mare a lui F poate încetini convergența, în timp ce una prea mică poate duce la blocarea în minime locale.

7. Bibliografie

1. Storn, R., & Price, K. (1997). *Differential Evolution – A Simple and Efficient Heuristic for Global Optimization*.
2. Suport de curs Inteligență Artificială.
3. Documentația NumPy și Matplotlib.

8. Listă contribuții membri echipă

Chihalau Gabriel-Eduard:

- Dezvoltarea structurii principale a algoritmului DE în model.py.
- Implementarea logicii de citire a datelor din CSV și prelucrarea lor pentru Problema 1.
- Elaborarea fundamentului teoretic și a bibliografiei în documentație.

Luigi Emanuel Arama:

- Implementarea simulatorului fizic pentru sistemul masă-arc-amortizor în `problem2.py`
- Proiectarea funcției de fitness pentru PID și a sistemului de penalizări.
- Realizarea testelor experimentale, generarea graficelor și redactarea concluziilor.